

COMP4610 / COMP8610 – Semester 1, 2026: Computer Lab #1

Declaration of Originality

We, Gusti Rais UID: 7962778, Muhammad Sean Arsha Galant UID: u7963630, Galih Ibrahim Kurniawan UID: 7954175 here declare that the code and report submitted in this assignment are my own work. The programs and results presented in this report were developed and written independently by me.

I confirm that I have **not copied code, text, or results from other students or external sources**, except where properly acknowledged. I understand that plagiarism or any form of academic dishonesty is not permitted.

AI Use: In completing this assignment, I declare that I:

☐ **Have not used any AI tools**

☒ **Have used AI tools** (e.g., ChatGPT, DeepSeek, or similar AI systems)

If you used AI tools, please briefly describe **how they were used**. For example:

- grammar or language editing
- help understanding the tasks and concepts
- debugging suggestions
- AI coding
- other assistance (please specify below)

Description of AI use (if applicable):

We used it for research and obtain research papers. We used it to understand concepts behind penguin waddling and movement. We also used it to improve writing (grammar, condense information, factcheck)

Student Signature (type your name): Gusti Rais, Muhammad Sean Arsha Galant, Galih Ibrahim Kurniawan

General Instructions

Please include this page as **the first page of your lab report** (this page does **not** count toward the page limit).

3D Character Animation & Interaction - Penguin Walk Cycle and Interactive Walking Environment

COMP4610 - Computer Graphics

Gusti Faturahman Rais - u7962778

Muhammad Sean Arsha Galant - u7963630

Galih Ibrahim Kurniawan - u7954175

Introduction

Our team chose the task to animate a 3d walk cycle as we believe it was the most appropriate project to implement lessons that we have learned in lab projects and lecture slides. Topics such as rasterization and hierarchy interested us in pursuing this task. We chose one of our lab 2 submissions for the cubism model used in this project. We wanted to implement an easy rigging and animation experience for this project, so we chose a bipedal penguin as the cubist animal for this project. Our other choices included a quadpedal cat and an octopoidal spider model. We believe that these discarded models would be entirely too much to animate.

Character Hierarchy and Joint Placement

The program for part 1 of this project is `main_1.cpp`, a real-time viewer for a penguin walk cycle animation built using SFML and a custom software rasterizer. The file coordinates loading, posing, rendering, and user interaction, and serves as the entry point that ties together several supporting modules.

When the program starts, it calls `loadPenguinObj` to read the geometry from a `.obj` file passed as a command-line argument. You will find that in the `assets` folder, we have `penguin_fixed.obj` and this will be the passed `obj` file to be rendered. This produces a flat `std::vector<Mesh>` where each entry holds vertex positions, face indices, and a flat color [1]. The file carries no structural information about how the pieces relate to one another. That structure is imposed entirely by the code that follows.

The `Animator::init(meshes)` call is where the flat list of meshes becomes an articulated figure. Each mesh group is matched to a joint index stored in `joints[j].meshIndex`, with joints that own no geometry holding a value of -1. These parent-child relationships between joints, such as the pelvis part parenting the two thighs are hardcoded in the skeleton logic and are not able to be reproduced with other `obj` files [2]. Each joint stores a local offset representing its position relative to the parent, chosen to sit at the pivot point of the part it controls.

To help with visualising the joints of our model, we have implemented an overlay that shows, in realtime, how they move. The overlay is toggled with the J key and is computed once at startup by `computeJointROMs` and then drawn each frame using `drawJointROMArcs`. This function reads the world-space position of each joint directly from `anim.sk.world` and projects it to screen space, drawing an arc that shows the angular range the joint sweeps through during the walk. Because the arcs are anchored to the same matrices that drive rendering, they remain correctly positioned relative to the penguin at all times.

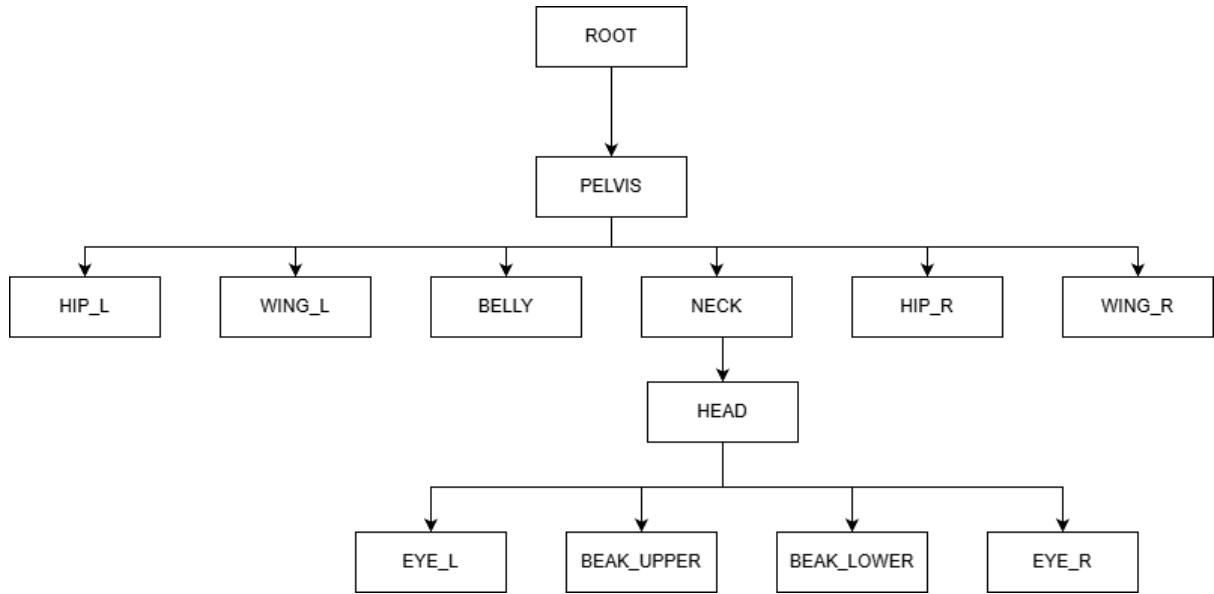


Figure 1: Penguin Model Hierarchy



Figure 2: Penguin Model

Rasterizer Method

Our renderer is a software rasterizer built from scratch in C++17, operating entirely on the CPU and delivering each completed frame to the display via a single SFML texture upload, with the following implementations:

Coordinate transform stack: Each mesh is stored in its own joint-local frame. Before rasterization, every vertex passes through four successive transforms: the joint's world matrix (built by the skeleton's forward-kinematics pass), a rigid model-to-world placement, the view matrix constructed from eye/center/up vectors using the standard look-at formulation, and a perspective projection that maps the view frustum to NDC. The resulting clip-space coordinate is divided by its w component and then scaled by the half-dimensions of the framebuffer to yield integer pixel

coordinates. The Y axis is flipped at this stage because screen space grows downward while NDC grows upward.

Backface culling: Before a triangle enters the rasterization loop it is tested against the camera. We compute the signed area of the projected triangle using the cross product of its two screen-space edge vectors. A negative signed area (clockwise winding in screen space, which arises from the NDC Y-flip applied to CCW-wound geometry) indicates a front-facing triangle; any triangle with non-negative area is discarded. This single comparison eliminates roughly half the input triangles with no per-pixel cost, thus saving a lot of compute time on triangles that aren't visible [2].

Edge-function rasterization: Front-facing triangles are rasterized using the Pineda edge-function algorithm [8]. For each triangle we compute the axis-aligned bounding box of its three vertices and iterate over every pixel centre within that box. Three scalar edge functions, one per triangle edge, evaluate a signed distance from the pixel to each edge; a pixel is inside the triangle when all three values share the same sign. The three edge-function values are also the barycentric weights of the pixel relative to the triangle's vertices, so depth and any other per-vertex attributes can be interpolated across the triangle face in the same pass.

Depth buffer: Occlusion is resolved with a per-pixel depth buffer following Catmull's algorithm [9]. Before the rasterizer writes a shaded colour to the framebuffer it interpolates the triangle's clip-space depth at the current pixel and compares it against the stored depth value. Only pixels whose interpolated depth is less than the stored value update both the colour buffer and the depth buffer. The depth buffer is cleared to 1.0 (the far-plane NDC depth) at the start of each frame.

Flat shading: Each triangle is assigned a single colour computed once from its face normal. We use a two-lobe model, a key light and a fill light, whose intensities were chosen following Millerson's three-point lighting ratios [10]:

$$I = 0.22 + 0.62 (N \cdot L_{\text{key}}) + 0.26 (N \cdot L_{\text{fill}}) \quad (1)$$

The constant 0.22 is an ambient floor. The face normal is computed in world space from the cross product of two edge vectors and normalised before the dot products are evaluated. Because the normal is constant across the triangle, all three dot products are evaluated once per triangle rather than once per pixel.

Background and ground plane: The background is a vertical gradient rasterized by a pre-pass that fills every pixel with a colour linearly interpolated between a sky colour at the top and a horizon colour at the bottom, before any geometry is drawn. The ground plane is rendered as a large flat quad placed at $y = 0$, shaded with the same flat shading method but using a fixed upward-facing normal, which produces a consistent matte appearance regardless of camera angle.

Frame delivery: Once the colour buffer is complete, the raw pixel data is uploaded to an SFML `sf::Texture` via a single `update(pixels)` call, drawn to a full-screen `sf::Sprite`, and presented with `window.display()`.

Animation Method and Key Designs

This project uses a procedural walk cycle system rather than keyframing. This means that every degree of freedom (DOF) is a closed-form function of a phase angle which is described as the following:

$$\phi = \frac{2\pi \cdot t}{T_{\text{period}}} \quad (2)$$

Such that T is the cycle period where $T_{period} = 1.10\text{ s}$, which gives the full walk cycle (or also known as a full gait cycle) sequence or two steps. This approach follows the parametric controller paradigm used in real-time games and biomechanics simulation; this is known as a cosine-family controller, avoids keyframe interpolation entirely and produces smooth, infinitely looping motion with zero state beyond the current time value [2] [3].

In terms of the animation related to the penguin's movement, it is notable that emperor penguins usually exhibit a lateral sway and waddling/leaning amplitude [4] [5]. In our model, the pelvis rolls about $\pm 22\text{ rad}$ at step frequency to simulate the lateral sway and waddling amplitude, with the roll phase timed so that the supporting foot lies directly beneath the center of mass; this is the inverted-pendulum walking model described by Griffin and Kram [4] as follows:

$$roll(t) = 0.22 \cdot \sin(\phi) \cdot s_r \quad (3)$$

where $s_r \in [0, 1]$ is the speed ratio.

For the vertical bob that applies to the penguin model, this is observable through each footstrike which pushes the center of mass upward. We model this as a double-frequency term, or two vertical bobs per cycle as the following. It should be noted that we used $|\sin|$ to ensure the bob is strictly non-negative so that the feet of the penguin never penetrate the ground plane:

$$bob(t) = 0.022 \cdot |\sin(2\phi)| \cdot s_r \quad (4)$$

For the leg swing, Griffin and Kram [4] note that penguin legs are short and stiff, which implies that the high metabolic cost comes from the short legs and not the waddle itself. Through testing, we found that our leg swing is quite modest ($\pm 35\text{ rad}$ at full speed) with a speed-dependent amplitude coupling, which according to Cavagna et al., the balance between vertical work (W_v) and forward kinetic work (W_f) shifts at different speeds [6]. This served as the basis of our following model for stride:

$$stride(t) = A_{stride} \cdot (0.45 + 0.55 \cdot s_r) \cdot \sin(\phi + offset) \quad (5)$$

The left and right legs are offset by π radians (anti-phase). Foot clearance (or lift) also uses a squared positive half-sine to prevent ground penetration (clipping through the ground floor) and to create a natural-looking snap in the movement of the penguin:

$$lift(t) = L \cdot [\max(0, \sin(\phi))]^2 \quad (6)$$

Penguins are also usually observed to show their flippers swinging opposite to their legs when walking, which provides angular momentum balance. Our wings for our penguin swing in anti-phase to the opposite legs of the penguin with an amplitude of $\pm 0.40\text{ rad}$ (or pitch) and adds a roll-plane flap at double the frequency for visual purposes. For the stabilisation of the penguin's head, research documents that birds stabilise their heads through counter-sway [7]. Our penguin's head yaws in about ± 4 degrees opposite to the body roll, and nods at a gait/subtle frequency just to add a detail of natural movement. When the penguin is stationary ($s_r = 0$), the model exhibits a gentle breathing-like body sway (0.035 rad at 0.7 Hz) and a periodic beak chatter (0.8 Hz), which blends out smoothly as the speed of the penguin increases.

All motion amplitudes scale with a speedRatio (variable) parameter (s_r) which is computed at each frame from the penguin's current velocity as follows:

$$s_r = \frac{|speed|}{v_{max}} \quad (7)$$

At rest ($s_r = 0$), the penguin is in idle mode, and at full speed ($s_r = 1$), all gait parameters are at their maximum. The speedRatio parameter modulates both amplitude and frequency of the gait. Stride amplitude grows linearly with speedRatio, matching the speed-amplitude coupling documented in biomechanics literature. When speedRatio is zero, the walk oscillations collapse to zero and a separate idle layer takes over, driving a slower breathing bob and gentle head sway using the always-advancing idleTime variable. This dual-layer approach means the beak chatter and breathing animations play continuously regardless of whether the penguin is moving or paused. It is notable that idle animations have an inverse blend factor $idle = \max(0, 1 - 4 \cdot s_r)$ that fades them out quickly once movement begins again, preventing overlap.

Layered on top of the continuous walk system are two discrete triggered actions: a backflip and a slide. Both are implemented as explicit state machines that apply their pose overrides after applyWalkPose completes each frame, and both force speedRatio to zero while active so that all walk oscillations are suppressed and the action has full control of the skeleton.

The backflip runs through six sequential phases: BF_CROUCH, BF_LAUNCH, BF_AIRBORNE, BF_DESCEND, BF_LAND, and BF_RECOVER, completing in approximately 1.45 seconds. Each phase drives a root height offset, a pelvis pitch rotation, and secondary pose adjustments for the legs and wings, all computed from a normalised phase progress value through cosine-based easing curves. The sequence begins with a 0.25-second crouch: the root drops 0.15 units via an easeInOut curve, the pelvis leans slightly forward by 0.3 rad as a windup, the wings begin to spread, and the legs begin to bend. In the 0.15-second launch phase the root accelerates upward to 30% of the peak height (0.54 units), the pelvis snaps from its forward lean into the start of the backward rotation, and the legs tuck to 0.7 rad. The main airborne phase lasts 0.40 seconds: the root follows a quadratic arc $h(t) = \text{JUMP_HEIGHT} \times (-4(t - 0.5)^2 + 1)$ peaking at 1.8 world units at the midpoint, while the pelvis pitch covers 5% to 80% of a full backward revolution (-2π rad) through an easeInOut curve, concentrating angular acceleration at the midpoint of the flip. The legs remain fully tucked and the wings hold their spread throughout. The 0.20-second descent phase completes the remaining 20% of the rotation via an easeOut curve as the root falls to a shallow landing crouch of 0.12 units and the legs extend for impact. BF_LAND then absorbs the impact over 0.15 seconds with a brief forward pelvis lean that fades out, and BF_RECOVER eases all residual offsets back to zero over 0.30 seconds.

The slide runs through four phases: SL_ENTER, SL_ACTIVE, SL_EXIT, and SL_IDLE. The SL_ENTER phase lasts 0.50 seconds and is split into two functional windows. During the first 45% the root joint is displaced upward by a sine arc, $\text{hopY} = \sin(\pi \cdot t) \times 0.30$, which peaks at the midpoint and returns to zero by landing; the body pitch blend is held at zero throughout so the penguin remains upright while airborne. In the remaining 55% the blend activates and eases in using a cosine-based curve, pitching the pelvis forward to 1.45 rad (approximately 83 degrees from vertical) while suppressing the walk roll. This two-window design produces a small hop followed by a belly-flop entry into the slide. The initial glide speed is set to the penguin's current walking speed plus a fixed

launch bonus of 2.0 units per second, so triggering the slide at a run carries noticeably more momentum than triggering it from a walk. SL_ACTIVE then holds the pose at full blend: the pelvis is nearly horizontal, the root has dropped 0.34 units so the belly skims the ground plane, the head tilts back 0.75 rad relative to the now-horizontal body, and both wings spread outward by 0.40 rad in the roll plane. Forward speed decelerates each frame at 1.4 units per second squared until the penguin comes to a stop. On exit, the blend eases back to zero over 0.40 seconds via the same easeInOut curve, returning the skeleton to its standing configuration.

Control Logic

The interactive loop runs at a target of 60 frames per second. Each iteration measures the time elapsed since the previous frame, referred to as the frame delta time (dt), and caps it at 0.05 seconds so that a stall or window drag does not produce a single enormous position jump. All input is handled through SFML's event and keyboard systems, which offer two distinct ways to read key state. Discrete actions such as toggling a mode or triggering an animation are handled through the event queue, which delivers one notification per physical key press regardless of how long the key is held. Continuous actions such as walking are handled by polling the keyboard state directly each frame, so that holding W produces movement every tick for as long as the key remains down.

Forward and backward movement are controlled by W and S, which accelerate and decelerate the penguin's speed at 4.0 and 6.0 units per second squared respectively, with the result kept within the current mode's maximum by clamping. A and D rotate the penguin in place at 2.2 radians per second. Rather than advancing the world position at a constant rate, the position update is modulated by a foot-contact term: $\text{footDrive} = 1 + \cos(\phi_2)$, where ϕ_2 runs at twice the stride frequency. This produces small forward impulses timed to each footstrike rather than a constant glide, which gives the locomotion a more grounded feel.

The F key toggles between walk and run modes. Both modes use the same speed-proportional animation rate described in the Animation section, but run mode applies an additional 2.5x multiplier on top of it, so the legs cycle visibly faster than speed alone would produce. The two modes also have different speed ceilings: 2.0 units per second for walk and 5.0 for run. When switching modes, the current speed is clamped to the new ceiling to prevent a momentary overspeed. The F key is ignored while the slide is active.

Triggered actions are assigned to B (backflip) and G (slide). The two actions are mutually exclusive: B is ignored while the slide is active, and G is ignored while the backflip is active. Pressing G while the slide is already running transitions it to SL_EXIT rather than re-triggering a new entry. The camera system offers a third-person follow camera on key 0 and five fixed-position cameras on keys 1 through 5. The fixed cameras hold their eye position frozen in world space but recompute their look-at direction each frame to keep the penguin centred as it moves. Space pauses and resumes the simulation, R resets the penguin to its origin with zero speed and default orientation, and J toggles the joint range-of-motion overlay.

Assumptions or simplifications

Several deliberate simplifications were made to keep the implementation tractable within the scope of the project. These are documented here to distinguish intentional design decisions from unimplemented features.

Rasterizer: Depth is interpolated linearly in NDC rather than with perspective-correct interpolation, where each vertex z is divided by its clip-space w value before barycentric blending. For the small, nearly axis-aligned cuboid geometry in this project the error is visually imperceptible, but the simplification would produce noticeable artefacts on large triangles viewed at oblique angles. No anti-aliasing, shadow mapping, or texture sampling is applied. Similarly, flat shading was implemented as opposed to Phong or Gouraud due to the simplicity of the cubism style model and its coloring. The shading model also uses two directional lights as a cheap approximation to environment lighting and the fill light direction is derived algorithmically from the key light rather than being independently spawned and positioned.

Animation: The walk cycle is entirely procedural and forward-kinematics-based. No inverse kinematics are used, which means the feet do not stay planted on the ground in a physically strict sense; instead, foot contact is approximated by the footDrive modulation term, which produces the timing of footstrikes without constraining the foot's world-space position.

Triggered actions: The backflip and slide are purely kinematic: the root trajectory and joint rotations are prescribed curves rather than the output of any physics simulation. The backflip's parabolic arc and the slide's friction deceleration are hand-authored approximations to the underlying dynamics. No collision detection is performed between body parts or between the penguin and the ground during these actions; the landing phases are timed to coincide with the root returning to ground level rather than being detected geometrically.

Sample Results



Figure 3: Camera Position 0, Follow Behind, while idle



Figure 4: Camera Position 1, Low Front, while walking

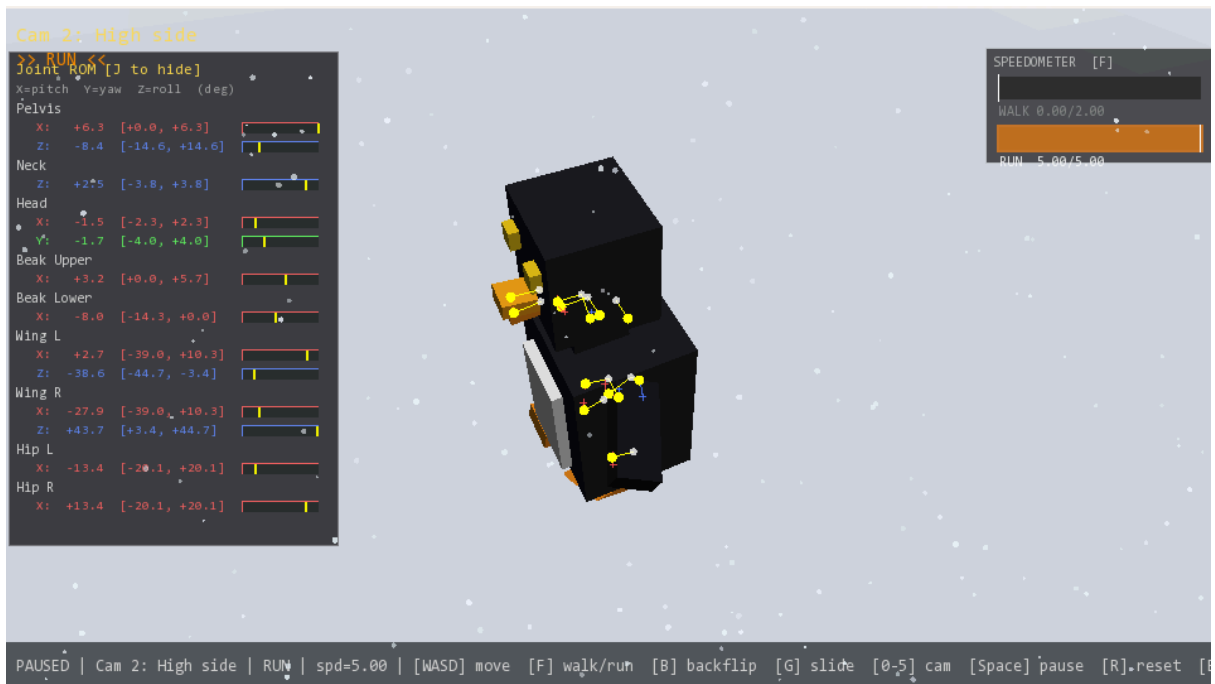


Figure 5: Camera Position 2, High Side, with Joint Range of Motion Overlay, while sprinting

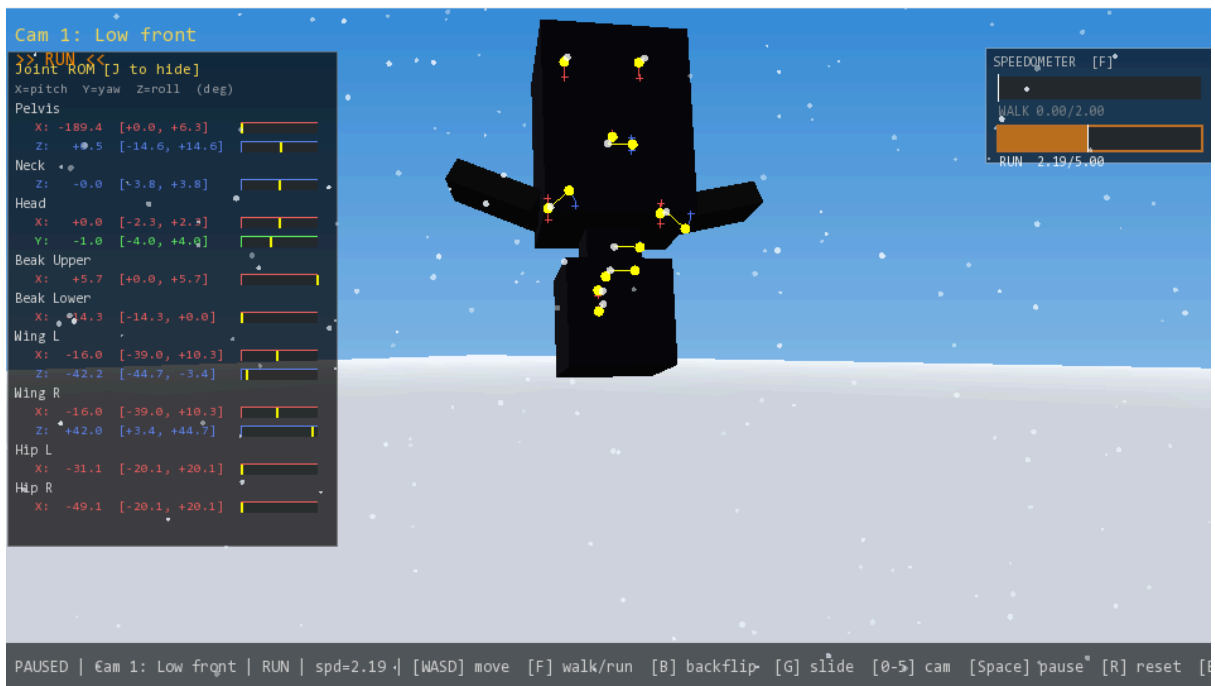


Figure 6: Camera Position 1, Low Front, with Joint Range of Motion Overlay, mid-backflip



Figure 7: Camera Position 1, Low Front, during slide

References

- [1] Wavefront Technologies, "Object Files (.obj)," Wavefront OBJ File Format Specification, Wavefront Technologies, Santa Barbara, CA, 1992.
- [2] T. Akenine-Möller, E. Haines, N. Hoffman, A. Pesce, M. Iwanicki, and S. Hillaire, Real-Time Rendering, 4th ed. Boca Raton, FL, USA: CRC Press, 2018.
- [2] A. Bruderlin and T. W. Calvert, "Goal-directed, dynamic animation of human walking," in Proc. SIGGRAPH '89, ACM, 1989, pp. 233–242.
- [3] L. Sun and D. Metaxas, "Automating gait generation," in Proc. SIGGRAPH '01, ACM, 2001, pp. 261–270.
- [4] T. M. Griffin and R. Kram, "Penguin waddling is not wasteful," Nature, vol. 408, pp. 929–930, Dec. 2000.
- [5] Willener AS, Handrich Y, Halsey LG, Strike S. Fat King Penguins Are Less Steady on Their Feet. PLoS One. 2016 Feb 17;11(2):e0147784. doi: 10.1371/journal.pone.0147784. PMID: 26886216; PMCID: PMC4757033.
- [6] G. A. Cavagna, H. Thys, and A. Zamboni, "The sources of external work in level walking and running," Journal of Physiology, vol. 262, pp. 639–657, 1976. \
- [7] R. Necker, "Head-bobbing of walking birds," Journal of Comparative Physiology A, vol. 193, pp. 1177–1183, 2007.
- [8] J. Pineda, "A parallel algorithm for polygon rasterization," ACM SIGGRAPH Computer Graphics, vol. 22, no. 4, pp. 17-20, 1988.
- [9] E. Catmull, "A subdivision algorithm for computer display of curved surfaces," Ph.D. dissertation, University of Utah, 1974.
- [10] G. Millerson, Lighting for Television and Film, 3rd ed. Focal Press, 1991.